SMART FARM MONITORING SYSTEM USING C

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0203- C PROGRAMMING

to the award of the degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING
&

ELECTRONS AND COMMUNICATION ENGINEERING

Submitted by

R. JAIDEEP (192525441)

B. BHANU PRAKASH (192412518)

P. JEEVAN KUMAR (192572090)

Under the supervision of

Dr. S. K. Saravanan

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

September 2026

Abstract

Modern agricultural farms contain many sensor nodes that continuously monitor important environmental parameters such as soil moisture, temperature, humidity, and water level. Managing these sensors manually becomes difficult as the number of sensor nodes increases.

The proposed Smart Farm Monitoring System using C provides a software-based approach for managing and monitoring multiple sensor records. The system supports dynamic sensor registration, pointer-based data manipulation, function-based processing, recursive traversal of a farm hierarchy, data storage, and inactive sensor detection.

The system periodically processes sensor information and identifies sensors that fail to provide data within a specified interval. When an inactive sensor is detected, the system generates an alert. A Tinker CAD-based hardware prototype is also used to demonstrate the practical monitoring concept using an Arduino, sensors, LCD, LEDs, and buzzer.

Keywords

Smart Farm, C Programming, Sensor Monitoring, Dynamic Memory, Recursion, File Handling, IoT, Inactive Sensor Detection

Introduction

Agriculture increasingly depends on technology to monitor environmental conditions and improve resource management. Sensors can provide continuous information about soil and environmental conditions, helping farmers make better decisions.

A large farm may contain hundreds of sensor nodes distributed across different fields. These sensors periodically provide information such as soil moisture, temperature, humidity, and water level.

As the number of sensors increases, manually monitoring every sensor becomes difficult. A monitoring system is therefore required to organize sensor records, process readings, maintain historical information, and identify sensors that stop communicating.

The proposed system uses C programming concepts to develop a prototype for smart farm sensor monitoring. It demonstrates dynamic memory allocation, structures, pointers, functions, recursion, file handling, and sensor-status analysis.

The system also provides an inactive sensor alert mechanism, which is the main feature of the hackathon challenge.

Problem Statement & Objectives

Problem Statement

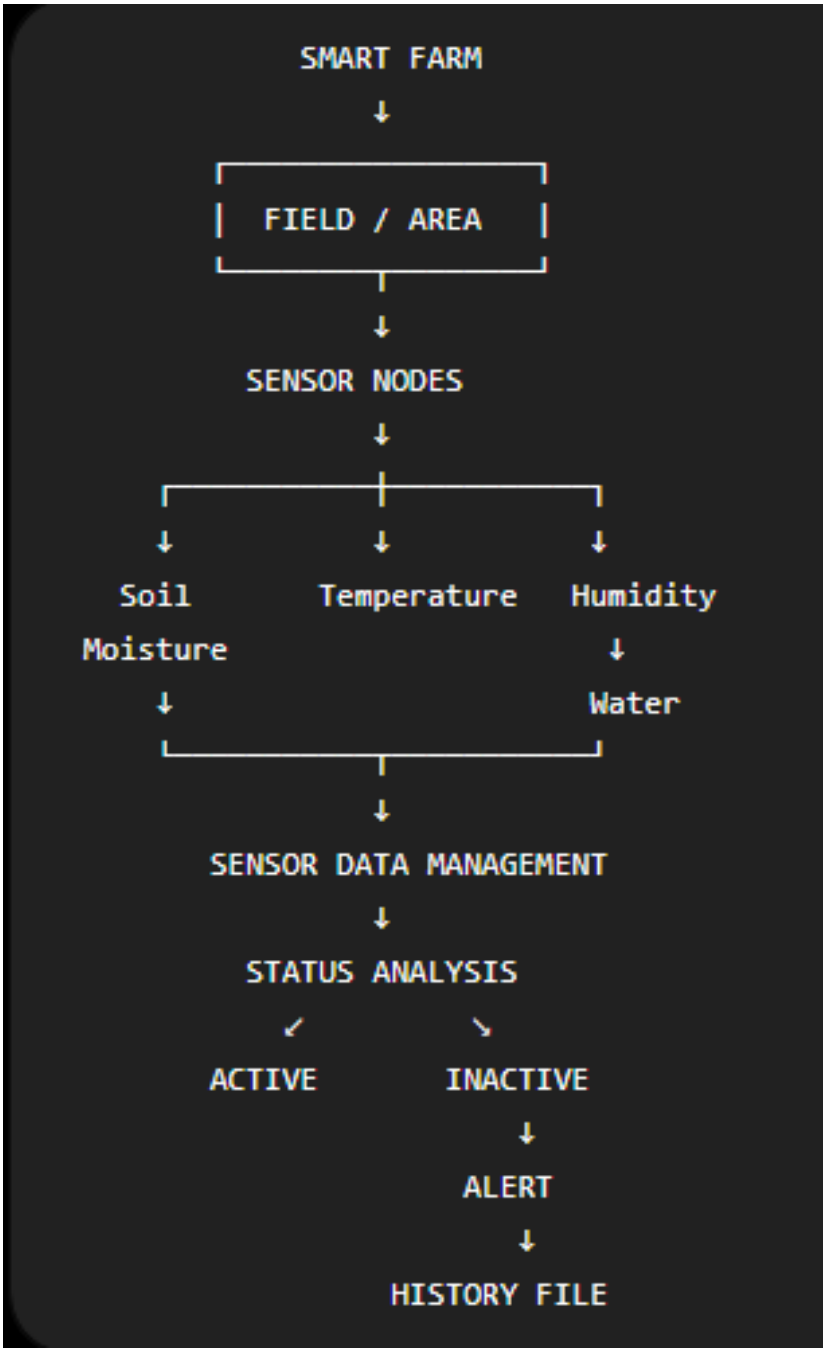
Large agricultural farms may contain hundreds of distributed sensor nodes. Sensors can occasionally stop transmitting information because of communication problems, power failure, hardware faults, or other issues. If inactive sensors are not identified quickly, the monitoring system may operate using incomplete or outdated information. Therefore, a system is required that can:

- Manage a dynamically increasing number of sensors.
- Store sensor information efficiently.
- Process sensor readings.
- Maintain historical sensor data.
- Monitor sensor activity.
- Detect inactive sensors.
- Generate automatic alerts.

Objectives:

- To dynamically create and manage sensor records.
- To use pointers for sensor data manipulation.
- To implement functions for sensor registration and analysis.
- To use recursion for hierarchical farm traversal.
- To simulate sensor data transmission.
- To store historical sensor readings in files.
- To identify inactive sensors.
- To generate an automatic alert for inactive sensors.
- To demonstrate the concept using a hardware simulation prototype.

System Architecture & Working



5. Working Principle

1. The system starts with the farm configuration.
2. Sensors are registered dynamically.
3. Each sensor is assigned a unique ID and type.
4. Sensor readings are received/generated periodically.
5. The system updates the sensor records.
6. Sensor activity is checked.
7. Active sensors continue normal operation.
8. If a sensor does not provide data within the specified interval, it is marked inactive.
9. An alert is generated for the inactive sensor.
10. Sensor readings are stored for historical reference.

Modules / Functionalities

Module 1 — Sensor Registration

- Allows new sensor nodes to be added dynamically. Each sensor contains an ID, sensor type, reading, and activity status.

Module 2 — Sensor Data Management

- Uses structures, pointers, and dynamic memory to manage sensor information.

Module 3 — Sensor Data Transmission

- Simulates periodic transmission of readings from multiple sensor nodes.

Module 4 — Farm Hierarchy

- Represents the farm structure using levels such as:

Farm → Field → Sensor Group

- Recursive traversal is used to process the hierarchy.

Module 5 — Historical Data Storage

- Sensor readings are stored in a file so that previous readings can be referenced later.

Module 6 — Inactive Sensor Detection

- The system checks whether sensor data has been received within the specified interval.

Module 7 — Alert Generation

- When an inactive sensor is detected, the system generates an alert identifying the sensor ID and sensor type.

C Programming Implementation

1.Structures: A structure is used to store sensor information.

Code:

```
typedef struct
{
int id;
char type[30];
float value;
int active;
} Sensor;
```

2. Dynamic Memory Allocation: Dynamic memory allows sensor records to increase as new sensors are registered.

Code:

```
sensors = realloc(
   sensors,
   (sensor_count + 1) * sizeof(Sensor)
);
```

Pointers: Pointers are used for dynamic sensor-data management.

Functions: Separate functions are used for:

- Sensor registration
- Data transmission
- Historical storage
- Inactive sensor checking
- Farm hierarchy traversal

Recursion: The farm hierarchy is traversed recursively.

File Handling

Sensor history is stored using file operations such as:

- fopen()
- fprintf()

fclose()

Command-Line Arguments: Field configuration can be provided through:

int main(int argc, char *argv[])

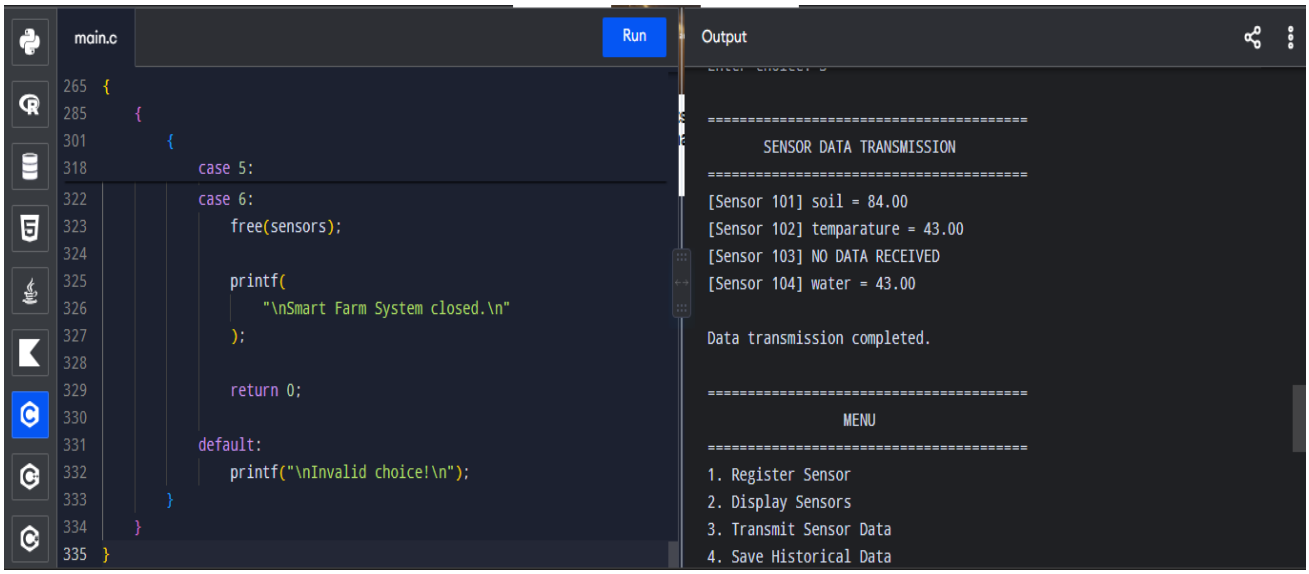
Results & Output

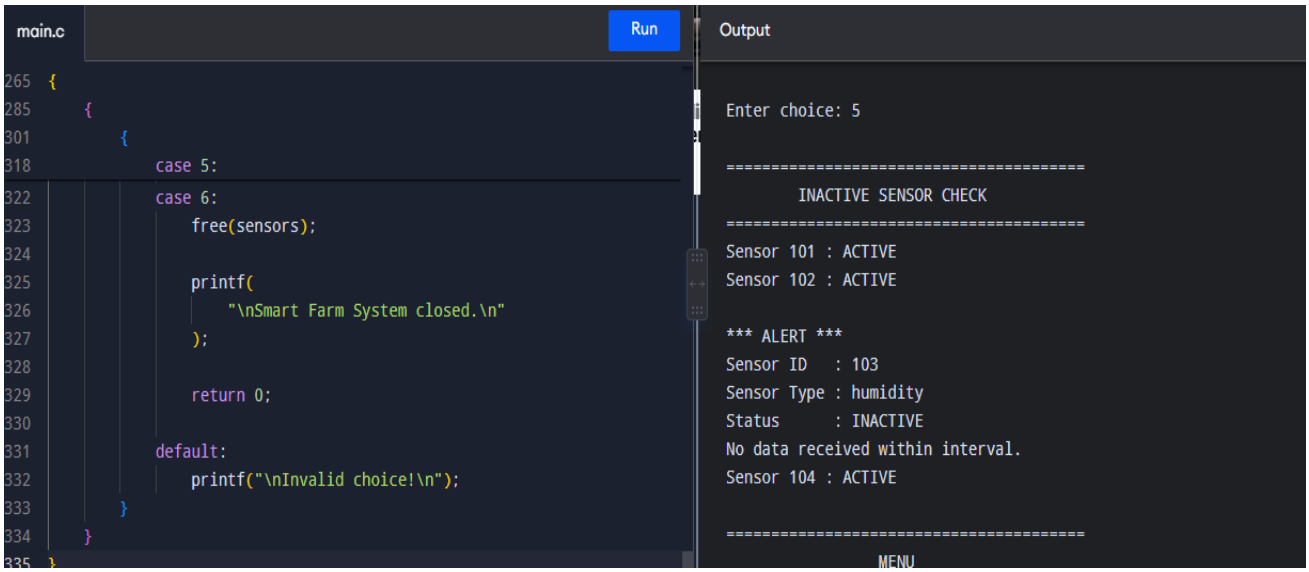
Code:

```
typedef struct
{
   int id;
   char type[30];
   float value;
   int active;
} Sensor;

void checkInactiveSensors()
{
   for(int i = 0; i < sensorCount; i++)
   {
      if(sensors[i].active == 0)
      {
         printf("ALERT: Sensor %d is inactive!\n",
               sensors[i].id);
      }
   }
}
```

Output:


main.c
Run
Output
case 5:
case 6:
free(sensors);
printf(
"\nSmart Farm System closed.\n"
);
return 0;
default:
printf("\nInvalid choice!\n");
SENSOR DATA TRANSMISSION
[Sensor 101] soil = 84.00
[Sensor 102] temparature = 43.00
[Sensor 103] NO DATA RECEIVED
[Sensor 104] water = 43.00
Data transmission completed.
MENU
1. Register Sensor
2. Display Sensors
3. Transmit Sensor Data
4. Save Historical Data


main.c
Run
Output
case 5:
case 6:
free(sensors);
printf(
"\nSmart Farm System closed.\n"
);
return 0;
default:
printf("\nInvalid choice!\n");
Enter choice: 5
INACTIVE SENSOR CHECK
Sensor 101 : ACTIVE
Sensor 102 : ACTIVE
*** ALERT ***
Sensor ID : 103
Sensor Type : humidity
Status : INACTIVE
No data received within interval.
Sensor 104 : ACTIVE
MENU

References

1. Kumar, V., Sharma, K. V., Kedam, N., Patel, A. B., Kate, T. R., & Rathnayake, U. (2024). A comprehensive review on smart and sustainable agriculture using IoT technologies. Smart Agricultural Technology, 8, 100487. https://doi.org/10.1016/j.atech.2024.100487
2. Shahab, H., Iqbal, M., Sohaib, A., Khan, F. U., & Waqas, M. (2024). IoT-based agriculture management techniques for sustainable farming: A comprehensive review. Computers and Electronics in Agriculture, 220, 108851. https://doi.org/10.1016/j.compag.2024.108851
3. Gatkal, N. R., Nalawade, S. M., Sahni, R. K., Bhanage, G. B., Walunj, A. A., Kadam, P. B., et al. (2024). Review of IoT and electronics enabled smart agriculture. International Journal of Agricultural and Biological Engineering, 17(5), 44–57. https://doi.org/10.25165/j.ijabe.20241705.8496
4. Bahadure, N. B., Joshi, R. R., Parashar, D. R., Shah, B., Patni, J. C., & Patil, P. D. (2025). Internet of things enabled smart agriculture monitoring and control system. In 2025 3rd International Conference on Disruptive Technologies (pp. 683–686). IEEE. https://doi.org/10.1109/ICDT63985.2025.10986450
5. Internet of things enabled smart agriculture: Status, latest advancements, challenges and countermeasures. (2025). Heliyon, 11(3), e42136. https://doi.org/10.1016/j.heliyon.2025.e42136
6. The IoT and AI in agriculture: The time is now—A systematic review of smart sensing technologies. (2025). Sensors, 25(12), 3583. https://doi.org/10.3390/s25123583
7. Virtual sensors for smart farming: An IoT- and AI-enabled approach. (2025). Internet of Things, 32, 101611. https://doi.org/10.1016/j.iot.2025.101611
8. Recent advances in IoT-driven crop monitoring and precision irrigation: Technologies, models, and future challenges. (2026). Scientific African, 31, e03222. https://doi.org/10.1016/j.sciaf.2026.e03222